

Manual Técnico Módulo 4



Os Fundos Europeus mais próximos de si.



Cofinanciado pela
União Europeia

Módulo 4

1. Programação Orientada a Objetos	3
1.1. Orientação a Objetos e Terminologia	3
1.2. Construtor e Parâmetro <i>self</i>	4
1.3. Atributos de Instância e de Classe	6
1.4. Validações no Construtor	7
1.5. Encapsulamento e Níveis de Acesso	8
1.6. Métodos Get e Set.....	9
1.7. @Property	10

1. Programação Orientada a Objetos

A transição para a Programação Orientada a Objetos (OOP) representa o segundo pilar estrutural do Módulo 4. Na primeira fase, a arquitetura funcional demonstrou ser a metodologia adequada para isolar lógicas de validação e orquestrar a execução linear de um *script* através de funções puras, controlo de escopo e gestão centralizada de exceções.

Contudo, o paradigma funcional apresenta limitações arquiteturais severas quando a ferramenta exige a manutenção de um estado contínuo em memória. Operações avançadas, como o rastreio simultâneo de múltiplas sessões de rede, a monitorização prolongada de anomalias ou a acumulação iterativa de dados forenses, exigem a retenção de dados complexos. A transferência constante de estruturas mutáveis (como dicionários e listas) entre funções independentes degrada a performance e expõe o sistema ao risco de contaminação cruzada de dados.

Esta segunda fase introduz a Programação Orientada a Objetos e o Encapsulamento como a solução de engenharia para este problema. O foco deixa de ser a execução sequencial de tarefas e passa para a modelação de entidades isoladas que blindam o seu próprio estado interno, garantindo a integridade dos dados durante todo o ciclo de vida da ferramenta.

1.1. Orientação a Objetos e Terminologia

A Programação Orientada a Objetos (OOP) é um paradigma de engenharia de software que organiza o código através do agrupamento de dados (estado) e comportamentos (ações) numa única estrutura.

A operacionalização deste paradigma exige o domínio da seguinte terminologia técnica:

- **Classe:** O molde estrutural abstrato. Define as propriedades e as regras de negócio, mas não ocupa espaço de dados na memória. (Exemplo: As características dum veículo).
- **Objeto (Instância):** A materialização da classe na memória RAM. A partir de uma única classe, o sistema aloca múltiplos objetos independentes.
- **Atributos:** As variáveis que pertencem a uma classe ou a um objeto. Representam o estado da entidade.
- **Métodos:** As funções definidas dentro de uma classe. Uma função atua isoladamente no *script*; um método atua exclusivamente sobre o estado do objeto ao qual pertence.

```
class Vehicle:
    # A Classe (o molde) dita que qualquer veículo gerado terá marca e modelo
    def __init__(self, brand: str, model: str):
        self.brand = brand
        self.model = model

    # O Método (a ação) utiliza o estado interno do próprio objeto (self)
    def show_identification(self) -> None:
        print(f"Veículo registado: {self.brand} {self.model}")

# Instanciação: Criação de dois Objetos independentes
work_car = Vehicle("Ford", "Focus")
personal_car = Vehicle("Fiat", "Punto")

# Cada objeto aciona o método utilizando o seu próprio espaço de memória
work_car.show_identification() # Output: Veículo registado: Ford Focus
personal_car.show_identification() # Output: Veículo registado: Fiat Punto
```

1.2. Construtor e Parâmetro *self*

A criação de um objeto exige a alocação de memória e a definição do seu estado inicial. Este processo é executado pelo método de inicialização obrigatório, designado por construtor (`__init__`).

O parâmetro *self* deve ser o primeiro parâmetro de qualquer método de instância. O *self* é a referência automática para o objeto específico que está a ser manipulado, garantindo que as alterações na memória não afetam outras instâncias da mesma classe.

self.brand vs brand

A instrução `self.brand = brand` exige a distinção de dois *scopes* de memória:

1. **O Parâmetro (brand):** É a variável temporária recebida no argumento do construtor. Existe estritamente no escopo local do método `__init__` e é destruída após a execução do construtor.
2. **O Atributo de Instância (self.brand):** Representa o espaço de armazenamento persistente alocado fisicamente no objeto. O prefixo `self.` indica que o dado pertence ao objeto e continuará a existir na memória.

A instrução `self.brand = brand` transfere o dado recebido pelo parâmetro para o atributo permanente da instância.

```
class Vehicle:
    def __init__(self, brand: str, model: str):
        # self.brand é o armazenamento permanente no objeto.
        # brand é a informação temporária recebida no momento da instanciação.
        self.brand = brand
        self.model = model
        self.current_speed = 0
```

Na construção de um objeto, pode ser vantajoso não obrigar o sistema a fornecer todos os dados de configuração. A linguagem permite definir parâmetros opcionais atribuindo um valor por defeito diretamente na assinatura do construtor (parametro: tipo = valor).

Para garantir a coerência dos dados e facilitar futuras manutenções, o valor de substituição (quando o parâmetro é omitido) não deve ser escrito diretamente no código lógico (*hardcoded*). A arquitetura correta exige a definição de uma constante privada ao nível da classe. Se o objeto for instanciado sem o dado, o construtor recorre a esta constante global para preencher o atributo.

```
class Vehicle:
    # Constante de Classe Privada: Define o valor padrão partilhado
    __DEFAULT_COLOR = "Branco"

    # O parâmetro 'color' é opcional.
    def __init__(self, brand: str, color = None):
        self.brand = brand

        # Lógica de fallback: Se a cor for None, vai buscar a constante privada
        if color is None:
            self.color = Vehicle.__DEFAULT_COLOR
        else:
            self.color = color

def main() -> None:
    # Instanciação fornecendo todos os argumentos
    custom_car = Vehicle("Ferrari", "Vermelho")
    print(custom_car.color) # Output: Vermelho

    # Instanciação omitindo o parâmetro opcional
    fleet_car = Vehicle("Renault")
    print(fleet_car.color) # Output: Branco

if __name__ == "__main__":
    main()
```

1.3. Atributos de Instância e de Classe

O modelo de dados em Python permite armazenar informação em dois níveis diferentes dentro da mesma arquitetura.

- **Atributos de Instância:** Declarados dentro do construtor (`__init__`) e prefixados com `self`. A memória é isolada e exclusiva de cada objeto.
- **Atributos de Classe:** Declarados no corpo da classe, fora de qualquer método. Não utilizam o `self`. Esta memória é partilhada globalmente por todos os objetos dessa classe.

A alteração de um atributo de classe afeta instantaneamente todos os objetos instanciados. Serve para definir configurações globais (ex: um limite de velocidade aplicável a todos os carros), sendo desadequado para armazenar estado dinâmico.

```
class Vehicle:
    # Atributo de Classe: Partilhado por todos os objetos
    legal_speed_limit = 120

    def __init__(self, brand: str):
        # Atributo de Instância: Exclusivo deste objeto
        self.brand = brand

print(Vehicle.legal_speed_limit)
# A modificação altera o limite para todos os veículos gerados
Vehicle.legal_speed_limit = 90
print()
print(Vehicle.legal_speed_limit)
```

1.4. Validações no Construtor

A arquitetura defensiva estabelece que um objeto não deve ser instanciado num estado inválido. A validação de dados tem de ocorrer no momento exato da criação.

Se um veículo for instanciado com uma potência negativa, a anomalia estrutural ocorre na raiz. O construtor deve conter lógica de validação para intercepar o erro e interromper a construção do objeto, levantando uma exceção antes da alocação de memória.

```
class Vehicle:
    def __init__(self, brand: str, engine_power: int):
        if engine_power <= 0:
            raise ValueError("O motor tem de possuir um valor de potência positivo.")

        self.brand = brand
        self.engine_power = engine_power
```

1.5. Encapsulamento e Níveis de Acesso

O acesso direto aos atributos (`car_one.engine_power = -50`) permite que módulos externos corrompam o estado interno do objeto. O encapsulamento restringe o acesso direto, forçando a passagem de dados por canais controlados.

O Python implementa esta proteção estrutural através de convenção de prefixos:

- **Público (`self.speed`):** Acesso totalmente livre.
- **Protegido (`self._speed`):** O prefixo com um *underscore* sinaliza que a variável é de uso interno. Não possui bloqueio técnico, baseando-se em convenção entre programadores.
- **Privado (`self.__speed`):** O duplo *underscore* ativa o *Name Mangling*. O interpretador renomeia dinamicamente a variável na memória (ex: `_Vehicle__speed`), bloqueando o acesso direto a partir de um módulo externo.

```
class Vehicle:
    def __init__(self, brand: str):
        # Nível Público: Acesso livre
        self.brand = brand

        # Nível Protegido: Acesso livre tecnicamente, mas restrito por convenção
        self._chassis_number = "CH-123456"

        # Nível Privado: Acesso bloqueado externamente via Name Mangling
        self.__max_speed = 200

def main() -> None:
    target_car = Vehicle("Renault")

    # SUCESSO: A leitura e escrita são permitidas no nível público
    print(target_car.brand)
    target_car.brand = "Peugeot"

    # SUCESSO: O interpretador permite, mas o programador está a violar a convenção
    print(target_car._chassis_number)

    # FALHA CRÍTICA: O interpretador gera um AttributeError. A variável está oculta.
    # print(target_car.__max_speed)
    # target_car.__max_speed = 250

if __name__ == "__main__":
    main()
```


1.6. Métodos Get e Set

A privatização do estado (`self.__speed`) impede a leitura ou alteração direta da variável. A solução clássica da engenharia de software consiste na criação de métodos públicos explícitos para interagir com o dado privado.

- **Método Get (Leitura):** Uma função desenhada para retornar o valor encapsulado.
- **Método Set (Escrita):** Uma função desenhada para receber um valor, aplicar validações lógicas e modificar o atributo privado apenas em caso de conformidade.

```
class Vehicle:
    def __init__(self):
        # Estado privatizado
        self.__speed = 0

    # Canal explícito de leitura
    def get_speed(self) -> int:
        return self.__speed

    # Canal explícito de escrita com validação
    def set_speed(self, target_speed: int) -> None:
        if target_speed < 0 or target_speed > 300:
            raise ValueError("Velocidade Impossível")

        self.__speed = target_speed

def main() -> None:
    car_one = Vehicle()

    # Manipulação através da invocação explícita de métodos
    car_one.set_speed(120)
    print(f"Velocidade atual: {car_one.get_speed()} km/h")
```

1.7. @Property

Embora a abordagem *Get* e *Set* cumpra os requisitos técnicos, a sintaxe nativa em Python utiliza decoradores para simplificar a interação com a classe.

O decorador `@property` mascara a execução dos métodos de validação. Permite que o código externo utilize a sintaxe simples de atribuição e leitura (`objeto.variavel`), enquanto o interpretador reencaminha essa ação para os métodos de controlo estrutural.

```
class Vehicle:
    def __init__(self):
        self.__current_speed = 0

    # O @property atua como o equivalente nativo do método Get
    @property
    def speed(self) -> int:
        return self.__current_speed

    # O @setter atua como o equivalente nativo do método Set
    @speed.setter
    def speed(self, target_speed: int) -> None:
        if target_speed < 0 or target_speed > 300:
            raise ValueError("Velocidade fisicamente impossível para a arquitetura.")

        self.__current_speed = target_speed

def main() -> None:
    car_one = Vehicle()
    # A sintaxe direta aciona o método @setter nos bastidores
    car_one.speed = 120

    # A chamada direta aciona o método @property
    print(f"Velocidade atual: {car_one.speed} km/h")
```